

A Digital Clockwork Simulator: Simulação de Circuitos Digitais Discretos em C++ com Suporte Multiplataforma

Francisco Passos dos S. Alves¹, João Vitor V. Leão¹, Karlos Danyel S. Gomes¹,
Lucca Pedreira Dultra¹

¹ Departamento de Computação – Universidade Federal de Sergipe (UFS)
São Cristóvão – SE – Brasil

{francisco.alves, joao.leao2, karlos.gomes2, lucca.dultra}@dcomp.ufs.br

Abstract. *This paper describes the development of A Digital Clockwork Simulator, a software simulator of a digital clock and alarm circuit, inspired by a discrete-logic hardware project by Wagner Rambo (WR Kits channel). The simulator was implemented in C++ following the object-oriented programming paradigm, with each integrated circuit modeled as an independent class that faithfully replicates the pin-level behavior described in the respective datasheet — including clock-edge dependencies, carry propagation, and asynchronous resets. The CD4000-series CMOS chips covered are: CD4029, CD4511, CD4017, CD4040, CD4013, CD4063, CD4081, and CD4071. As a course extension, a complete alarm subsystem (The Amazing Digital Alarm) was developed, integrating cascaded magnitude comparators and D flip-flops for programmed-time storage and real-time comparison. The project also adopts a cross-platform architecture for keyboard input and audio output, with one implementation file per operating system (Linux, Windows, and macOS) compiled selectively by the Makefile behind platform-agnostic interfaces. The simulator is distributed under the GNU General Public License (GPL) and is publicly available on GitHub.*

Resumo. *Este artigo descreve o desenvolvimento de A Digital Clockwork Simulator, um simulador de software de um circuito de relógio digital e alarme, inspirado em um projeto de hardware com lógica discreta de Wagner Rambo (canal WR Kits). O simulador foi implementado em C++ seguindo o paradigma de Programação Orientada a Objetos, com cada circuito integrado modelado como uma classe independente que replica fielmente o comportamento em nível de pino descrito no respectivo datasheet — incluindo dependências de borda de clock, propagação de carry e resets assíncronos. Os chips CMOS da série CD4000 abrangidos são: CD4029, CD4511, CD4017, CD4040, CD4013, CD4063, CD4081 e CD4071. Como extensão da disciplina, foi desenvolvido um subsistema de alarme completo (The Amazing Digital Alarm), integrando comparadores de magnitude cascadeados e flip-flops D para armazenamento de horário programado e comparação em tempo real. O projeto adota ainda uma arquitetura multiplataforma para captura de teclado e saída de áudio, com um arquivo de implementação por sistema operacional (Linux, Windows e macOS) compilado seletivamente pelo Makefile, por trás de interfaces agnósticas de plataforma. O simulador é distribuído sob a licença GNU General Public License (GPL) e está disponível publicamente no GitHub.*

1. Introdução

O estudo de sistemas digitais compreende a transição fundamental entre a abstração da álgebra booleana e a implementação física de circuitos eletrônicos [?]. Projetos que exigem a integração de componentes discretos — contadores, decodificadores, flip-flops e comparadores — são particularmente eficazes para consolidar essa transição, pois obrigam o estudante a raciocinar simultaneamente sobre sinais individuais, bordas de *clock* e propagação de estados ao longo de uma cadeia de chips.

A *Digital Clockwork Simulator* foi desenvolvido com esse objetivo: reproduzir, em software, o comportamento lógico de um relógio digital de lógica discreta projetado por Wagner Rambo e disponibilizado em seu canal no YouTube, WR Kits [?]. O simulador foi implementado em linguagem C++ utilizando o paradigma de Programação Orientada a Objetos (POO), modelando cada circuito integrado como uma classe independente cujos métodos correspondem diretamente às operações descritas nos *datasheets* dos respectivos componentes.

A abordagem adotada caracteriza-se pela fidelidade ao nível de pino do hardware: não são introduzidas abstrações de software que não encontrem equivalente no circuito físico. Se o hardware exigiria dois pulsos de *clock* sequenciais em chips distintos, a simulação aplica exatamente dois pulsos sequenciais. Essa escolha, além de pedagogicamente relevante, torna o código diretamente rastreável ao esquemático original, facilitando a compreensão da relação entre o comportamento lógico simulado e o comportamento elétrico real.

Como extensão do escopo original, requisitada pela disciplina de Sistemas Digitais, o projeto incorpora um subsistema de alarme completo — denominado *The Amazing Digital Alarm* — que integra comparadores de magnitude cascadeados, flip-flops D e lógica de validação de dia da semana. O conjunto foi ainda evoluído para suporte multiplataforma (Linux, Windows e macOS), mantendo o módulo de simulação inteiramente independente do sistema operacional hospedeiro.

O projeto é distribuído sob a licença GNU General Public License (GPL) e está disponível publicamente no GitHub¹.

2. Fundamentação Teórica e Componentes

A família CMOS CD4000 reúne circuitos integrados de baixo consumo, ampla faixa de tensão de alimentação e comportamento lógico bem definido em *datasheets* públicos [?]. Essa padronização torna os componentes da série especialmente adequados para fins educacionais: cada pino possui função inequívoca, e a cascata entre chips obedece a regras de temporização estáveis. No simulador, cada CI foi implementado como uma classe C++ em `include/chips.hpp` e `src/chips.cpp`, com os métodos públicos correspondendo diretamente aos pinos e operações do *datasheet*.

O CD4029 [?] é um contador BCD *Up/Down* presetable de 4 bits, empregado na contagem das unidades e dezenas de minutos e horas. O mecanismo de *ripple carry* foi modelado para que cada contador subsequente receba o pulso de *Carry In* apenas quando o anterior gerar *Carry Out* — replicando fielmente a cascata que ocorreria entre chips físicos e que, segundo [?], é a base da contagem multidígito em sistemas BCD.

¹<https://github.com/FrankSteps/digital-clockwork-simulator>

O **CD4511** [?] é o decodificador BCD para display de sete segmentos responsável por converter as saídas binárias dos CD4029 nos sinais que ativam cada segmento do display renderizado no terminal. A implementação reproduz os três pinos de controle definidos no *datasheet*: *Lamp Test* (LE), *Blanking* (BL) e *Latch Enable* (LE), permitindo que o display possa ser habilitado, apagado ou congelado independentemente da contagem em andamento.

O **CD4017** [?] é um contador Johnson de dez estágios que, no projeto, é instanciado de duas formas distintas: no módulo de relógio, configurado com limite de reset em 2 para funcionar como indicador de meridiano (AM/PM); e no módulo de alarme, configurado com limite de reset em 7 para implementar o contador de dias da semana. Essa reutilização do mesmo componente em funções diferentes ilustra a flexibilidade proporcionada pela configuração programática do limite de contagem.

O **CD4040** [?] é um divisor de frequência ripple de 12 estágios. No simulador, recebe o sinal de 60 Hz produzido pelo `FreqGenerator` e fornece, em suas saídas Q_n , subfrequências binárias decrescentes que são utilizadas nos modos de ajuste lento e rápido do relógio. A combinação de duas saídas do CD4040 por portas AND do CD4081 produz o pulso que avança os contadores no modo padrão de operação.

O **CD4013** [?] é um flip-flop D duplo de borda positiva. Conforme descrito em [?], flip-flops D são os elementos básicos de armazenamento em sistemas digitais síncronos: a saída Q assume o valor da entrada D somente na borda ativa do *clock*, permanecendo estável entre bordas. No módulo de alarme, 9 instâncias do CD4013 (18 flip-flops individuais) armazenam o horário programado: 16 bits de tempo em BCD (HH:MM), 1 bit de meridiano e 1 bit de estado de armamento (*stand-by*).

O **CD4063** [?] é um comparador de magnitude de 4 bits com entradas de cascata (*Equal*, *Greater*, *Smaller*). Cinco instâncias são cascadeadas no módulo de alarme para comparar o horário armazenado nos flip-flops com o barramento de tempo atual, propagando os sinais de comparação da unidade de minutos até o meridiano — exatamente como seria feito em hardware físico para comparações de palavras maiores que 4 bits [?].

O **CD4081** [?] (quatro portas AND de 2 entradas) e o **CD4071** [?] (quatro portas OR de 2 entradas) são empregados em duas funções distintas: na lógica de reset assíncrono dos contadores de horas, detectando condições de overflow específicas; e na redução lógica do comparador de dias da semana, em que sete sinais AND individuais (um por dia) são combinados por uma cascata de portas OR até produzir um único sinal de validação.

2.1. Gerador de Frequência

No projeto original de Wagner Rambo [?], o *clock* do relógio é derivado diretamente da rede elétrica brasileira (60 Hz): um transformador reduz a tensão da rede para o nível lógico de operação, e um circuito de condicionamento de sinal converte a senoide resultante em um sinal digital de onda quadrada, aproveitando simultaneamente a frequência e a tensão da rede sem necessidade de um oscilador dedicado [?].

No simulador, esse papel é desempenhado pela classe `FreqGenerator` (`include/freqGenerator.hpp`, `src/freqGenerator.cpp`). Uma *thread* dedicada alterna o sinal lógico a cada meio período de 60 Hz, utilizando

`std::this_thread::sleep_until` ancorado em pontos de tempo absolutos — estratégia que compensa atrasos acumulados ao longo da execução e reduz a deriva temporal em relação a abordagens baseadas em `sleep_for`. O sinal é entregue diretamente como onda quadrada, uma vez que sinais senoidais não possuem representação em lógica digital discreta: a conversão que o circuito de condicionamento realizaria no hardware é assumida como pré-existente na abstração do simulador.

3. Arquitetura e Implementação

O simulador é organizado em dois grandes módulos de simulação interligados: o *A Digital Clockwork*, responsável pela geração e manutenção do tempo, e o *The Amazing Digital Alarm*, responsável pelo armazenamento do horário programado, pela comparação com o tempo atual e pelo disparo do alarme. A Figura ?? apresenta a interface do simulador em execução no terminal (Linux).

3.1. O Relógio — *A Digital Clockwork*

O módulo de relógio é implementado na classe `DigitalClockwork` (`include/digitalClockwork.hpp`, `src/digitalClockwork.cpp`). O `FreqGenerator` fornece o sinal de referência de 60 Hz, que é aplicado ao CD4040. Esse circuito integrado atua como um divisor de frequência ripple, produzindo em suas saídas sinais cujas frequências correspondem à frequência de entrada dividida por potências de dois. Essas saídas são combinadas pelas portas AND do CD4081 para gerar os pulsos responsáveis por avançar os contadores CD4029 nos diferentes modos de operação.

A contagem de tempo é realizada por quatro instâncias do CD4029, dispostas em cascata: unidade de minutos, dezena de minutos, unidade de horas e dezena de horas. O mecanismo de *ripple carry* entre contadores é modelado no método `cascadeCount()`: o CD4029[0] (unidade de minutos) recebe o pulso de *clock* diretamente; os demais só avançam se o anterior gerou *Carry Out*, replicando o comportamento de cascata descrito em [?] para contadores multidígito.

Os resets assíncronos — necessários para reiniciar a dezena de minutos ao atingir 6 e para reiniciar as horas ao atingir 13 e retornar a 1 — são detectados pelo método `updateAND()`, que lê combinações específicas dos bits de saída dos CD4029 e aciona as portas AND do CD4081[0]. Quando a condição é satisfeita, `applyResets()` aplica o pulso de *Preset Enable* ao contador correspondente — replicando o comportamento de reset assíncrono por realimentação de saídas, amplamente documentado em projetos de relógio digital com lógica discreta [?]. A Figura ?? ilustra o diagrama esquemático dessa lógica.

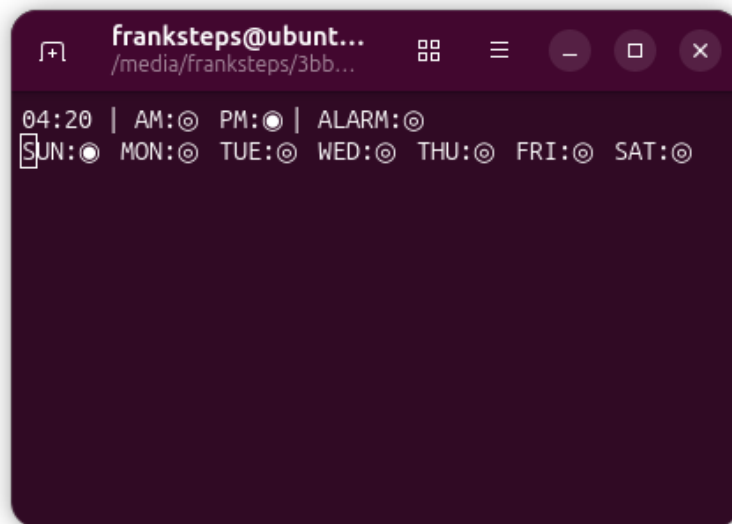


Figura 1. Interface do *A Digital Clockwork Simulator* renderizada no terminal (Linux).

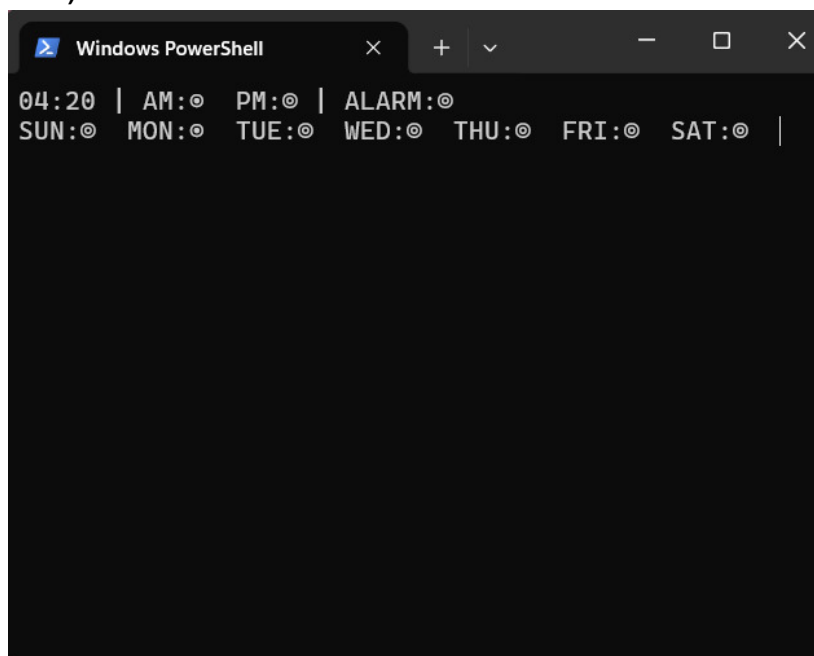


Figura 2. Interface no terminal (Windows).

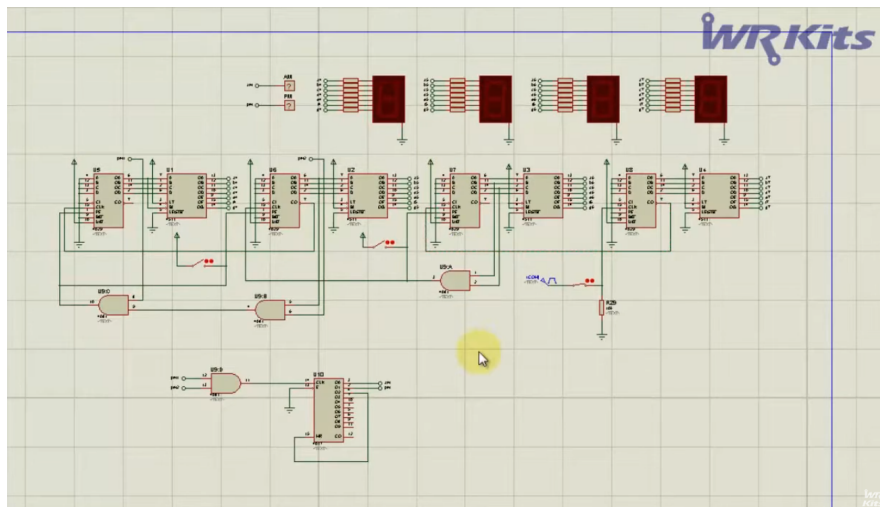


Figura 3. Diagrama esquemático do módulo de contagem do relógio digital.

A seleção da frequência de avanço do contador segundo o modo de operação (padrão, lento ou rápido) é realizada pelo método `frequencyConverter()`, apresentado na Listagem ?? . No modo padrão (DEFAULT), a saída combinada das portas AND do CD4081[1] produz um pulso por minuto; nos modos de ajuste, as saídas Q_0 e Q_5 do CD4040 fornecem, respectivamente, frequências mais elevadas para ajuste rápido e mais reduzidas para ajuste lento. A detecção de borda — condição $Q \ \&\& \ !lastQ$ — garante que o contador avance apenas uma vez por transição positiva, evitando múltiplos incrementos por nível lógico alto.

```

1 void DigitalClockwork::frequencyConverter(ADJUSTMENT MODE) {
2     bool Q0 = cd4040.getOutput(0);
3     bool Q5 = cd4040.getOutput(5);
4
5     updateAND_1();
6     bool Out4081 = cd4081[1].getOutput(2);
7
8     switch(MODE) {
9         case ADJUSTMENT::FAST:
10             if(Q0 && !lastQ0){
11                 updateCounter();
12             }
13             break;
14         case ADJUSTMENT::SLOW:
15             if(Q5 && !lastQ5){
16                 updateCounter();
17             }
18             break;
19         case ADJUSTMENT::DEFAULT:
20             if(Out4081 && !lastOut4081 && defaultCooldown == 0){
21                 updateCounter();
22                 defaultCooldown = 500;
23             }
24             if(defaultCooldown > 0) defaultCooldown--;
25             break;
26     }
27     lastQ0 = Q0;

```

```

28     lastQ5 = Q5;
29     lastOut4081 = Out4081;
30 }

```

Listing 1. Seleção de frequência por modo de ajuste em DigitalClockwork.

3.2. O Alarme — *The Amazing Digital Alarm*

O módulo de alarme é implementado na classe `DigitalAlarm` (include/digitalAlarm.hpp, src/digitalAlarm.cpp), integrando lógica combinacional e sequencial em torno de três caminhos independentes: memória de horário, comparação de tempo e comparação de dia da semana. O esquema lógico estrutural completo é apresentado na Figura ??.

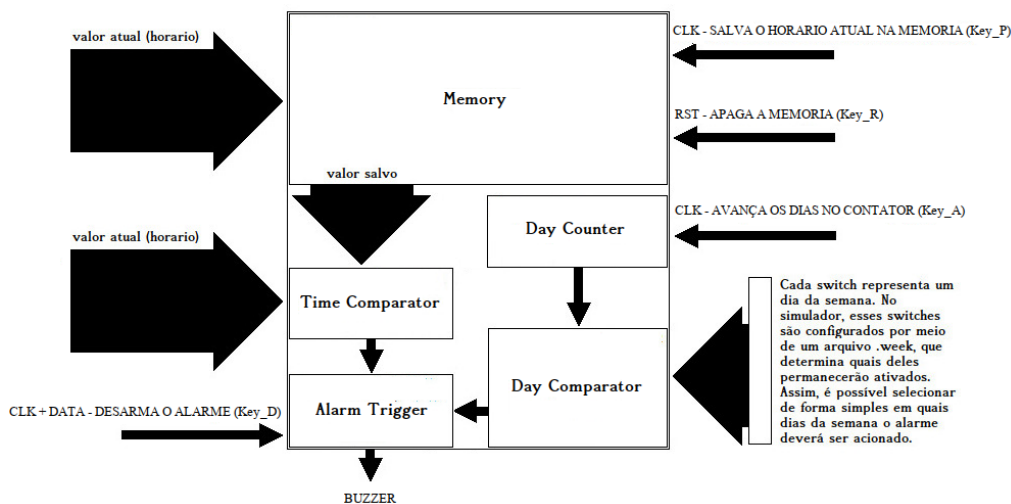


Figura 4. Esquema lógico estrutural do *Amazing Digital Alarm*.

Fonte: Autoria própria.

3.2.1. Memória de Horário

O armazenamento do horário programado utiliza 9 instâncias do CD4013 [?] — 18 flip-flops D individuais — distribuídas da seguinte forma: 16 bits para o tempo HH:MM em BCD (bits 0 a 15), 1 bit de meridiano (bit 16) e 1 bit de estado de armamento (*stand-by*), no flip-flop 1 do CD4013[8]. A separação entre a atualização das entradas D e a captura efetiva pela borda de *clock* é um aspecto central da implementação: o método `setDataMemory()` atualiza continuamente as entradas D de todos os flip-flops a cada ciclo, sem aplicar pulsos, mantendo o barramento de tempo atual “à porta” da memória; apenas quando o método `programAlarm()` é invocado (tecla P) os pulsos de *clock* são aplicados simultaneamente a todos os flip-flops, capturando o horário exibido naquele instante. Esse comportamento replica diretamente o mecanismo de *latch* descrito em [?] para flip-flops D em modo de captura síncrona. A Figura ?? ilustra o caminho de dados da memória.

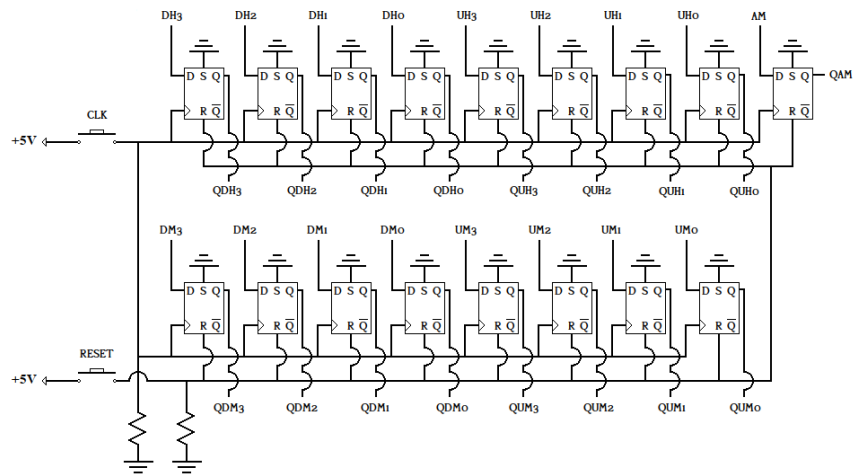


Figura 5. Diagrama do módulo de memória do alarme (CD4013).

Fonte: Autoria própria.

3.2.2. Comparação de Tempo

A comparação entre o horário armazenado nos flip-flops e o barramento de tempo atual é realizada por uma cascata de cinco instâncias do CD4063 [?]. O CD4063[0] compara a unidade de minutos; CD4063[1], a dezena de minutos; CD4063[2] e CD4063[3], a unidade e a dezena de horas; CD4063[4] compara o bit de meridiano. Em cada estágio, as saídas *Equal*, *Greater* e *Smaller* do estágio anterior são conectadas às entradas de cascata do estágio seguinte — o mesmo procedimento descrito em [?] para extensão da comparação a palavras maiores que 4 bits. O sinal de igualdade final do CD4063[4] indica que o horário programado coincide com o tempo atual. A Figura ?? detalha esse caminho.

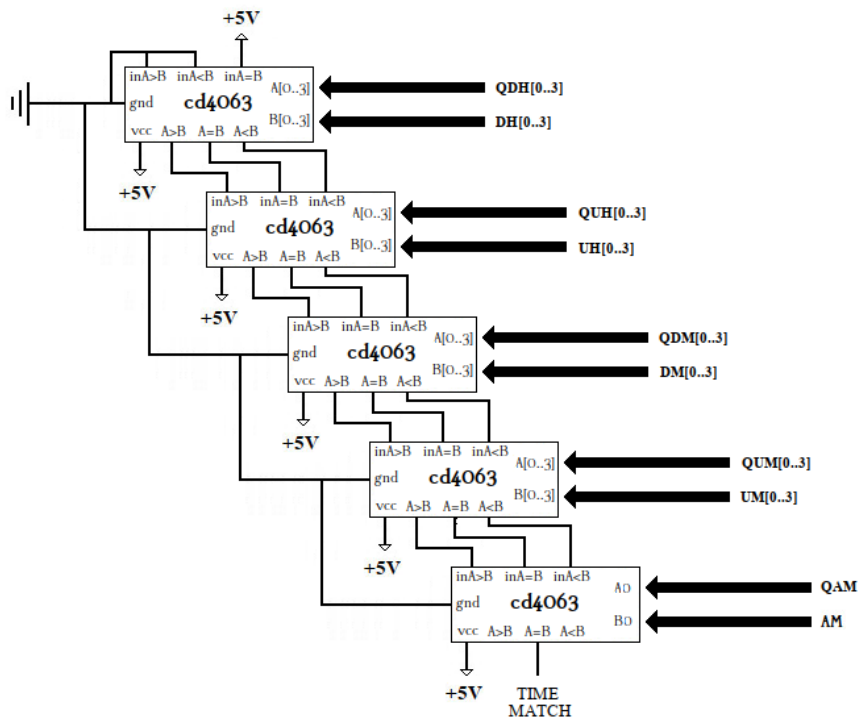


Figura 6. Cascata de comparadores de magnitude CD4063 para validação de horário.

Fonte: Autoria própria.

3.2.3. Comparação de Dia da Semana

A configuração dos dias em que o alarme deve ser ativado é realizada por meio de um arquivo `.week`, que simula sete chaves DIP físicas ON/OFF — uma por dia da semana. O método `loadWeekFile()` realiza a leitura e validação rigorosa do arquivo: são rejeitados arquivos com extensão incorreta, linhas malformadas, valores fora do domínio $\{0, 1\}$ e dias declarados fora da ordem esperada (SUN a SAT). A validação de dia no alarme combina a saída ativa do CD4017 (indicando o dia corrente) com o valor da chave correspondente do arquivo `.week` por meio de portas AND do CD4081; em seguida, uma cascata de portas OR do CD4071 reduz os sete sinais AND a um único sinal de “dia habilitado”. A Figura ?? ilustra essa lógica.

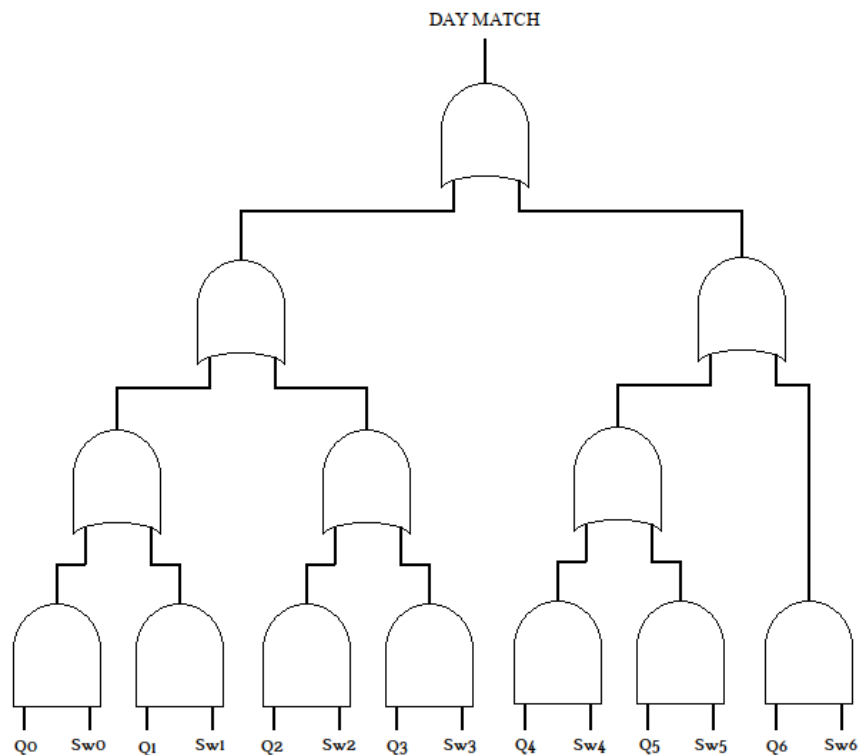


Figura 7. Lógica de comparação de dia da semana com CD4081 e CD4071.

Fonte: Autoria própria.

3.2.4. Disparo e Desarmamento

A saída final do alarme resulta da combinação, por uma porta AND do CD4081[2], de três sinais independentes: o sinal de “dia habilitado”, o sinal de igualdade de tempo e o sinal de *stand-by* (flip-flop 1 do CD4013[8]). O resultado é então combinado com o sinal Q_6 do CD4040, proveniente do *Digital Clockwork*, produzindo um pulso rítmico que aciona o Buzzer de forma intermitente. O desarmamento é realizado pela tecla D (*Disarm*), que aplica reset no flip-flop de *stand-by*, interrompendo a saída; a tecla R (*Reset*) apaga integralmente a memória, aplicando reset em todos os 18 flip-flops. A Figura ?? ilustra o circuito de disparo e desarmamento.

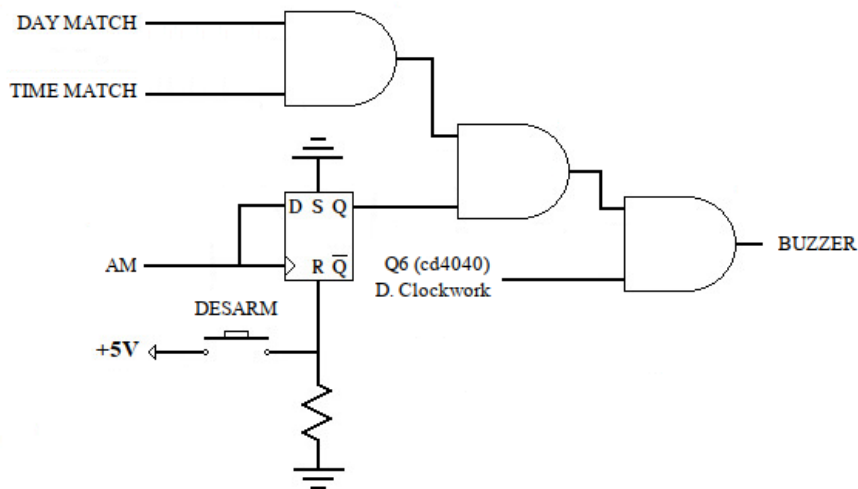


Figura 8. Circuito de disparo e desarmamento do alarme.

Fonte: Autoria própria.

3.3. Interface de Terminal e Entrada de Teclado

Todo o estado lógico simulado é refletido em tempo real no terminal do sistema operacional pelas classes `Display` e `Led` (`include/feedback.hpp`, `src/feedback.cpp`). O `Display` converte as saídas dos CD4511 em dígitos de sete segmentos renderizados em caracteres ASCII; os indicadores `Led` exibem o meridiano (AM/PM), o dia ativo no CD4017 e o estado de armamento do alarme por meio dos símbolos `•` (ligado) e `◦` (desligado).

A captura de teclado segue o mesmo princípio arquitetural aplicado aos demais módulos dependentes de plataforma: uma interface comum declarada em `include/keyboard.hpp` expõe as funções `initKeyboard()`, `pollKeys()` e `closeKeyboard()`, implementadas separadamente em `src/keyboard_linux.cpp` (via `libevdev`, lendo eventos brutos do kernel em `/dev/input/eventX`), `src/keyboard_windows.cpp` (via `GetAsyncKeyState` da WinAPI, com *polling* global independente de foco de janela) e `src/keyboard_macintosh.cpp` (via `CGEventTap` de *ApplicationServices*, *event tap* em nível de sistema). O `main.cpp` invoca exclusivamente as funções da interface e permanece idêntico nas três plataformas.

3.4. Saída de Áudio

O Buzzer, declarado em `include/audio_output.hpp`, segue a mesma arquitetura de interface comum com implementação por plataforma. No Linux, `src/audio_output_linux.cpp` utiliza ALSA (`libasound`) com escritas bloqueantes via `snd_pcm_writei`; no Windows, `src/audio_output_windows.cpp` utiliza WinMM (`waveOut*`) com buffers do tipo `WAVEHDR`; no macOS, `src/audio_output_macintosh.cpp` utiliza CoreAudio `AudioQueue`, cujo modelo orientado a *callback* difere estruturalmente das abordagens bloqueantes das outras plataformas, embora a interface pública do Buzzer permaneça idêntica.

O sinal sonoro gerado é uma onda quadrada de 2000 Hz pré-calculada no construtor: o `soundBuffer` alterna entre os valores `+32767` e `-32767` (máximo e mínimo de um *short* de 16 bits com sinal) a cada meio período, produzindo o bipe característico de um buzzer eletrônico. Uma *thread* dedicada alterna continuamente entre a escrita do `soundBuffer` e do `silenceBuffer` (todo zeros) conforme o estado do sinal lógico recebido via `setState()`. O controle de concorrência entre a *thread* do buzzer e o restante do programa é realizado com `std::atomic<bool>` — escolha intencional em relação ao `mutex`, pois as variáveis de controle (`running` e `state`) são lidas e escritas de forma isolada, sem necessidade de atomicidade entre operações múltiplas, tornando a instrução atômica de CPU a solução correta e mais eficiente para esse caso.

3.5. Seleção de Plataforma no Makefile

A seleção dos arquivos de implementação específicos de cada plataforma é realizada em tempo de compilação pelo *Makefile*, que detecta o sistema operacional via `uname -s` em Linux e macOS e via a variável de ambiente `$(OS)` em Windows. Com base nessa detecção, as variáveis `KEYBOARD_SRC`, `AUDIO_SRC` e `CONSOLE_SRC` recebem os arquivos corretos, e as *flags* de linkagem são ajustadas de forma correspondente: `-lasound` e `-levdev` no Linux; `-framework AudioToolbox`, `-framework Carbon` e `-framework ApplicationServices` no macOS; e `-lwinmm` no Windows. O `main.cpp` permanece intocado e idêntico nas três plataformas — a portabilidade é resolvida inteiramente na camada de build, sem condicionais de pré-processador no código principal.

4. Conclusão

O desenvolvimento do *A Digital Clockwork Simulator* demonstrou que a simulação orientada a objetos de circuitos digitais discretos constitui uma abordagem pedagogicamente eficaz para o estudo de sistemas digitais. A modelagem de cada CI como uma classe C++ independente, com métodos que refletem diretamente os pinos e operações definidos nos *datasheets* [?, ?, ?, ?, ?, ?, ?, ?], exigiu compreensão aprofundada tanto do comportamento elétrico de cada componente quanto dos mecanismos de temporização e cascata que governam o funcionamento do circuito como um todo [?, ?].

A extensão com o *Amazing Digital Alarm* forçou a integração de lógica combinacional e sequencial em escala significativa: 18 flip-flops D distribuídos em 9 instâncias do CD4013, 5 comparadores de magnitude CD4063 cascadeados e uma rede de 4 chips CD4071 e CD4081 atuando em conjunto para produzir um único sinal de disparo bem definido. A implementação bem-sucedida desse subsistema evidencia que o domínio da abstração de arquiteturas digitais permite transpor diretamente a lógica de circuitos físicos para sistemas de software estruturados.

Por fim, a evolução do projeto para suporte multiplataforma — com implementações separadas por sistema operacional para teclado, áudio e configuração de terminal, todas compiladas seletivamente pelo *Makefile* por trás de interfaces comuns — demonstrou que é possível manter um núcleo de simulação completamente agnóstico de plataforma sem recorrer a hierarquias de classes ou a condicionais de pré-processador no código principal. A portabilidade foi resolvida na camada de *build*, preservando a legibilidade e a manutenibilidade do código de simulação.

Referências

- TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L. *Sistemas Digitais: Princípios e Aplicações*. 11. ed. São Paulo: Pearson Prentice Hall, 2011.
- FLOYD, T. L. *Fundamentos de Sistemas Digitais*. 9. ed. São Paulo: Pearson Prentice Hall, 2007.
- MAMEDE FILHO, J. *Manual de Equipamentos Elétricos*. 4. ed. Rio de Janeiro: LTC, 2012.
- RAMBO, W. *Relógio Digital com Lógica Discreta CMOS*. Canal WR Kits (YouTube), 2015. Disponível em: <https://www.youtube.com/@WRKits>.
- TEXAS INSTRUMENTS. *CD4029B Types: CMOS Presettable Up/Down Counter — Datasheet*. 2003. Disponível em: <https://www.ti.com/lit/ds/symlink/cd4029b.pdf>.
- TEXAS INSTRUMENTS. *CD4511B Types: CMOS BCD-to-7-Segment Latch/Decoder/Driver — Datasheet*. 2003. Disponível em: <https://www.ti.com/lit/ds/symlink/cd4511b.pdf>.
- TEXAS INSTRUMENTS. *CD4017B Types: CMOS Decade Counter/Divider — Datasheet*. 2003. Disponível em: <https://www.ti.com/lit/ds/symlink/cd4017b.pdf>.
- TEXAS INSTRUMENTS. *CD4040B Types: CMOS 12-Stage Ripple-Carry Binary Counter/Divider — Datasheet*. 2003. Disponível em: <https://www.ti.com/lit/ds/symlink/cd4040b.pdf>.
- TEXAS INSTRUMENTS. *CD4013B Types: CMOS Dual D-Type Flip-Flop — Datasheet*. 2003. Disponível em: <https://www.ti.com/lit/ds/symlink/cd4013b.pdf>.
- TEXAS INSTRUMENTS. *CD4063B Types: CMOS 4-Bit Magnitude Comparator — Datasheet*. 2003. Disponível em: <https://www.ti.com/lit/ds/symlink/cd4063b.pdf>.
- TEXAS INSTRUMENTS. *CD4081B Types: CMOS Quad 2-Input AND Gate — Datasheet*. 2003. Disponível em: <https://www.ti.com/lit/ds/symlink/cd4081b.pdf>.
- TEXAS INSTRUMENTS. *CD4071B Types: CMOS Quad 2-Input OR Gate — Datasheet*. 2003. Disponível em: <https://www.ti.com/lit/ds/symlink/cd4071b.pdf>.